

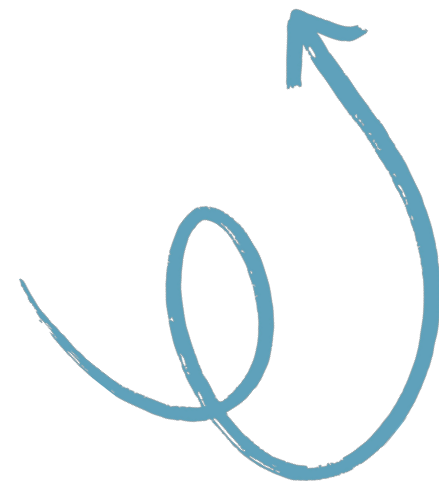
React Native

para

Aplicaciones

móviles

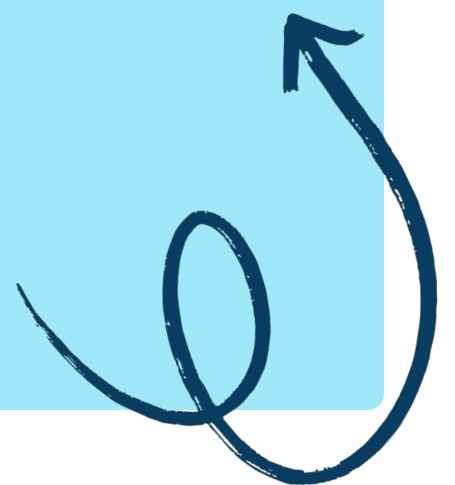
Yanina A. Dacal Villamayor



¿Qué es React Native?

Framework para aplicaciones móviles multiplataforma

React Native es un **framework innovador** que permite crear aplicaciones móviles para Android e iOS utilizando **JavaScript** y componentes reutilizables, optimizando el **desarrollo** y mejorando la eficiencia.



Cómo funciona

Estructura de una app móvil

Pantallas

Las aplicaciones móviles se estructuran en **pantallas** que permiten a los usuarios interactuar. Cada pantalla tiene un propósito, ofreciendo características y funcionalidades específicas para mejorar la experiencia del usuario.

Componentes UI

Los **componentes de UI** son elementos reutilizables que componen la interfaz. Incluyen botones, textos e imágenes, facilitando la creación de aplicaciones intuitivas y visualmente atractivas que mantienen la coherencia.



Componentes Básicos

View



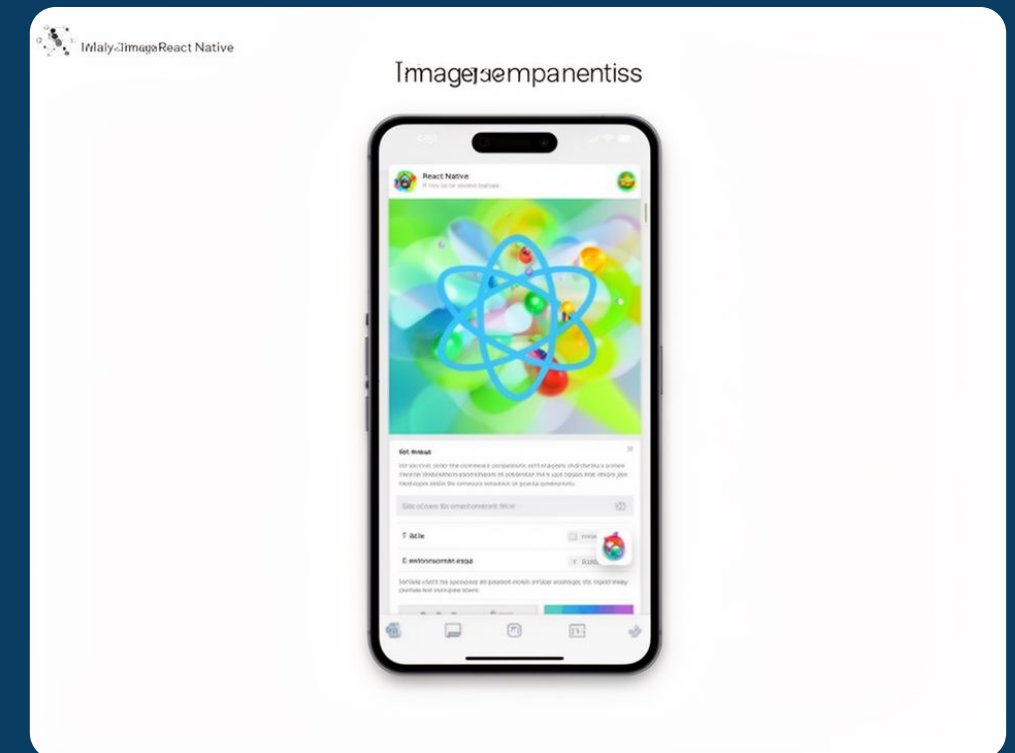
View es el contenedor principal utilizado para organizar (disponer) los elementos de la interfaz de usuario en aplicaciones de React Native

Text



El componente Text muestra texto en la pantalla, admitiendo varios estilos y propiedades.

Image



El componente Image permite a las aplicaciones mostrar imágenes, mejorando la interacción y la experiencia del usuario.

Botones en React Native

Función y uso en navegación

Los botones en React Native son esenciales para la **navegación entre pantallas**. Permiten al usuario interactuar con la aplicación, facilitando acciones como enviar datos, realizar compras y regresar a pantallas anteriores.

```
<Button  
  title="Presioname"  
  onPress={() => alert('¡Click!')}  
/>
```

Explicación:

- **title** → texto visible
- **onPress** → acción al tocar

Limitación:

No se puede personalizar mucho el diseño.



Componente Button

Implementación básica en React Native

Función del botón

El componente Button es esencial para la **interacción del usuario**. Permite activar acciones, como navegar entre pantallas o enviar información, mejorando la experiencia general de la aplicación.

Código de ejemplo

El siguiente código muestra cómo implementar un botón simple: `title="Presioname" y onPress={() => alert('¡Click!')}`. Esto genera una alerta al ser presionado, mostrando su funcionalidad básica.

Limitaciones del diseño

A pesar de su simplicidad, el diseño del botón en React Native es **poco personalizable**. Para estilos más avanzados, se pueden considerar alternativas como `TouchableOpacity`, que ofrecen mayor flexibilidad.



Propiedades del Botón

Elementos clave para personalización efectiva

```
<Button  
title="Aceptar"  
color="green"  
onPress={() =>  
console.log('OK')}  
>
```

Título

El **título** del botón es lo que los usuarios verán; debe ser claro y descriptivo para que comprendan la acción que van a realizar al presionarlo.

onPress

La propiedad **onPress** define la acción que se ejecuta cuando el botón es presionado, permitiendo conectar la interfaz con la lógica de la aplicación de manera efectiva.

Color

La propiedad **color** permite personalizar el aspecto visual del botón, asegurando que se alinee con el diseño general de la aplicación y mejore la experiencia del usuario.



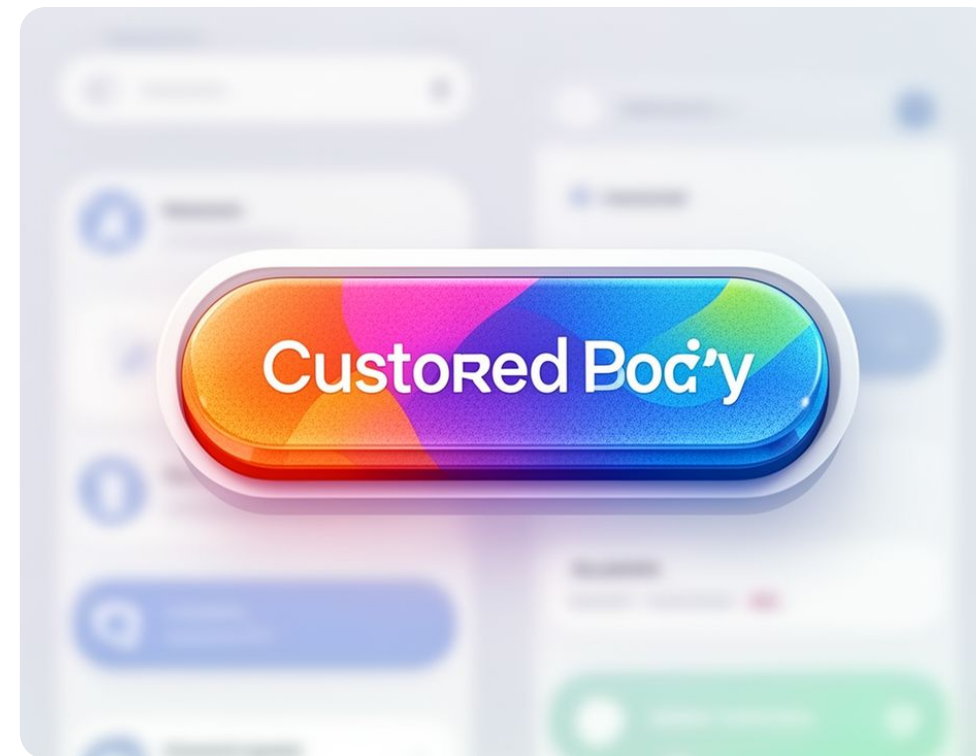
Botones Personalizados

TouchableOpacity



TouchableOpacity permite crear botones **interactivos** con control sobre su apariencia y respuesta al usuario.

Estilos Personalizables



Los botones personalizados mejoran la **experiencia del usuario** al ofrecer un diseño atractivo y funcional.

Experiencia del Usuario



Integrar botones personalizados **ofrece** una experiencia más rica y atractiva para los usuarios de la aplicación.

Estilos Personalizados

Mejora la apariencia de los botones

```
<TouchableOpacity
style={{ backgroundColor: 'blue', padding: 10 }}
onPress={() => alert('Click')}
>
  <Text style={{ color: 'white' }}>Botón
  lindo</Text>
</TouchableOpacity>
```

Diseño Atractivo

Los botones con estilos personalizados permiten una **mejor experiencia de usuario** y pueden adaptarse a la estética general de la aplicación, destacando la marca y funcionalidad.

Flexibilidad Visual

Utilizar componentes como TouchableOpacity proporciona un **mayor control** sobre la apariencia, permitiendo ajustes de color, tamaño y estilo que se alineen con el diseño deseado.

Separación de Lógica

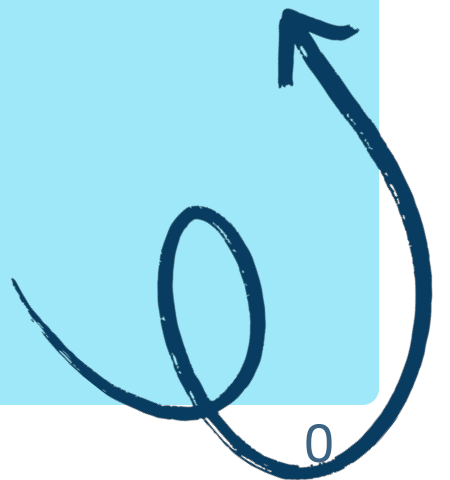
Al aplicar estilos en un objeto JavaScript, se logra una **organización clara**, manteniendo la lógica de código separada de la presentación visual, lo que facilita el mantenimiento y la escalabilidad.



Navegación en Apps

Clave para una experiencia fluida

La **navegación en aplicaciones móviles** es esencial para permitir a los usuarios moverse entre diferentes pantallas, facilitando la interacción y mejorando la usabilidad de la app.



Navegación en Pila

Pantallas apiladas para fácil acceso

La **navegación en pila** permite manejar múltiples pantallas, donde la última vista es la que se visualiza. Facilita la experiencia del usuario al navegar entre diferentes secciones de la app.

 **Ejemplo real:**

- **Inicio → perfil → configuración → volver**

 **Importante:**

Sin navegación, la app sería una sola pantalla.

Navegación en Pila

Avanzando entre pantallas en apps

navigation.push('Details');

Concepto de Push

La función navigation.push permite agregar una nueva pantalla **encima de la pila** de navegación, facilitando la transición a una vista diferente en la aplicación móvil.

Funcionalidad de Navegación

Esta técnica es crucial para **mejorar la experiencia** del usuario, permitiéndole acceder a información adicional y regresar fácilmente a la pantalla anterior sin perder su contexto.

Ejemplo Práctico

Un uso común sería un botón que, al presionarlo, activa navigation.push('Details'), llevando a los usuarios a la pantalla de detalles de un elemento seleccionado.

Navegación hacia atrás

Cómo volver a la pantalla anterior

`navigation.goBack();`

Función de retroceso

La función de retroceso permite que los usuarios **naveguen fácilmente** hacia la pantalla anterior, mejorando la experiencia general de la aplicación. Es un elemento clave en la navegación móvil.

Uso de código

Para implementar esta función, utilizamos el código `navigation.goBack();`. Este comando elimina la pantalla actual y regresa a la anterior en la pila de navegación.

Importancia de la navegación

Sin una navegación adecuada, la **interacción del usuario** se vuelve confusa. El retroceso permite a los usuarios explorar sin perder su lugar, haciendo que la app sea más intuitiva.

Ejemplo combinado

Relación entre botón y navegación

Navegación sencilla

La navegación sencilla permite al usuario **transitar entre pantallas** de manera intuitiva. Un botón puede ser el elemento clave para activar este cambio, mejorando la experiencia general.

Código práctico

En este ejemplo, el siguiente código usa un botón para llevar al usuario a una nueva pantalla:

```
onPress={() =>
  navigation.push('Details')},
mostrando la funcionalidad directa
del botón.
```

Integración completa

Integrar la navegación mediante botones es esencial en **aplicaciones móviles**. Esta técnica asegura que los usuarios puedan acceder a diferentes partes de la app con facilidad y rapidez.

Reproducción de Multimedia

Integrando sonidos y videos en apps

La **reproducción de audio y video** en aplicaciones permite una **experiencia interactiva** enriquecedora. Con librerías adecuadas, los desarrolladores pueden agregar música y sonidos en sus proyectos para mejorar la interacción.

```
<Button title="Reproducir" onPress={playSound}  
/>
```

Estilos en React Native

Definición de estilos con JavaScript

Objetos JavaScript

En React Native, los estilos se definen como **objetos JavaScript**, lo que permite una integración más profunda con la lógica de la aplicación y facilita su mantenimiento.

Flexibilidad

Esta metodología brinda **flexibilidad** al diseñar interfaces, permitiendo la adaptación rápida a cambios en el diseño sin la necesidad de alterar múltiples archivos CSS.

Ejemplo de estilo

Un ejemplo de estilo en React Native es definir un objeto que contenga propiedades como `backgroundColor` y `padding`, que se aplican directamente a los componentes.

Flexbox en React Native

Organización y alineación de elementos

Flexbox es una herramienta poderosa que permite **distribuir y alinear** componentes de forma eficiente en la pantalla. Su uso optimiza el diseño, mejorando la **experiencia del usuario** en aplicaciones móviles.



Styled Components

Estilos en React Native facilitados

Ventajas

Los **Styled Components** permiten definir estilos de forma modular, manteniendo una buena organización en el código y mejorando la legibilidad de los componentes dentro de la aplicación.

Sintaxis

Utilizan una sintaxis similar a CSS, lo que facilita su adopción por parte de desarrolladores familiarizados con el diseño web, permitiendo una transición fluida entre ambos mundos.

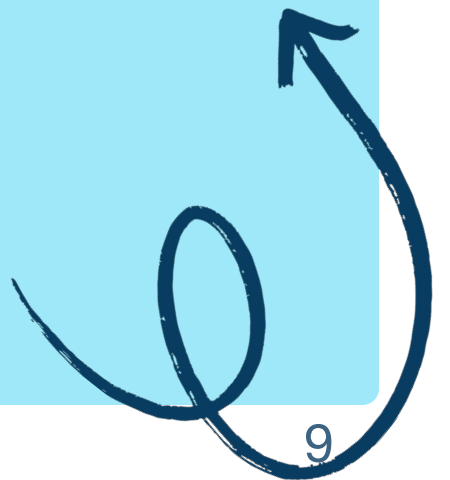
Ejemplo

Por ejemplo, se puede crear un componente de texto estilizado de la siguiente manera: `const Texto = styled.Text \font-size: 20px;`;` siendo accesible y fácil de entender.

Ventajas de React Native

Beneficios clave para desarrolladores

React Native permite **desarrollar aplicaciones móviles** eficientemente gracias a su código reutilizable, mejor organización del proyecto y acceso a una amplia comunidad que apoya el desarrollo.



Resumen de conceptos clave



01

Componentes

Los **componentes** son la base de una aplicación React Native. Permiten estructurar la interfaz y reutilizar el código, facilitando el desarrollo de aplicaciones móviles eficientes.

02

Botones

Los **botones** son esenciales para la interacción del usuario. Permiten la navegación entre pantallas y la ejecución de acciones, mejorando la experiencia general de la aplicación móvil.